

Commander.js Build Status		Install Size gitterjoin chat	The complete solution for node.js command-line interfaces, inspired by Ruby’s commander. API documentation Installation \$ npm install commander Option parsing Options with commander are defined with the .option() method, also serving as
1 8 <pre>#!/usr/bin/env node the value of program.sauce to false. following word. For example, --no-sauce sets --no prefix negate the boolean value of the Note that multi-word options starting with program.templateEngine etc. Multi-word options such as “-template- for example -abc is equivalent to -a -b -c. Short flags may be passed as a single arg,</pre>	2 7 <pre>.parse(process.argv); console.log('you ordered a pizza with:'); if (program.peppers) console.log(' - peppers'); if (program.pineapple) console.log(' - pineapple'); if (program.bbqSauce) console.log(' - bbq'); console.log(' - %s cheese', program.cheese);</pre>	3 9 <pre>program .version('0.1.0') .option('-p, --peppers', 'Add peppers') .option('-P, --pineapple', 'Add pineapple') .option('-b, --bbq-sauce', 'Add sauce') .option('-c, --cheese [type]', 'Add the specified type of cheese [marble]', 'marble');</pre>	4 5 <pre>documentation for the options. The example below parses args and options from process.argv, leaving remaining args as the program.args array which were not consumed by options. #!/usr/bin/env node /** * Module dependencies. */ var program = require('commander');</pre>
6 9 <pre>/** * Module dependencies. */ var program = require('commander'); program .option('--no-sauce', 'Remove sauce') .parse(process.argv); console.log('you ordered a pizza');</pre>	7 10 <pre>if (program.sauce) console.log(' with sauce'); else console.log(' without sauce');</pre> To get string arguments from options you will need to use angle brackets <> for required inputs or square brackets [] for optional inputs. e.g. .option('-m --myarg [myVar]', 'my super cool description') Then to access the input if it was passed in. e.g. var myInput = program.myarg	8 11 NOTE: If you pass a argument without using brackets the example above will return true and not the value passed in. Version option Calling the version implicitly adds the -V and --version options to the command. When either of these options is present, the command prints the version number and exits.	9 12 <pre>\$./examples/pizza -V 0.0.1</pre> If you want your program to respond to the -v option instead of the -V option, simply pass custom flags to the version method using the same syntax as the option method. <pre>program .version('0.0.1', '-v, --version')</pre> The version flags can be named anything, but the long option is required.
10 16 <pre>function increaseVerbosity(v, total) { return total + 1; } program .version('0.1.0') .usage('[options] <file ...>') .option('-i, --integer <n>', 'An integer argument', parseInt)</pre>	11 15 Coercion <pre>function range(val) { return val.split('').map(Number); } function list(val) { return val.split(','); } function collect(val, memo) { memo.push(val); return memo; }</pre>	12 14 <pre>console.log('remove ' + dir + (cmd.recursive ? ' recursively' : '')) } program.parse(process.argv) A command's options are validated when the command is used. Any unknown options will be reported as an error. However, if an action-based command does not define an action, then the options are not validated.</pre>	13 13 <pre>#!/usr/bin/env node You can attach options to a command. var program = require('commander'); program .command('rm <dir>') .option('-r, --recursive', 'Remove recursively') .action(function (dir, cmd) {</pre>

<pre> .option('-f, --float <n>', 'A float argument', parseFloat) .option('-r, --range <a>.', 'A range', range) .option('-l, --list <items>', 'A list', list) .option('-o, --optional [value]', 'An optional value') .option('-c, --collect [value]', 'A repeatable value', collect, []) </pre>	<pre> .option('-v, --verbose', 'A value that can be increased', increaseVerbosity, 0) .parse(process.argv); console.log(' int: %j', program.integer); console.log(' float: %j', program.float); console.log(' optional: %j', program.optional); program.range = program.range []; </pre>	<pre> console.log(' range: %j..%j', program.range[0], program.range[1]); console.log(' list: %j', program.list); console.log(' collect: %j', program.collect); console.log(' verbosity: %j', program.verbosity); console.log(' args: %j', program.args); </pre>	Regular Expression <pre> program .version('0.1.0') .option('-s --size <size>', 'Pizza size', /^(large medium small) \$/i, 'medium') .option('-d --drink [drink]', 'Drink', /^(coke pepsi izze)\$/ i) .parse(process.argv); </pre>
Specify the argument syntax <pre> #!/usr/bin/env node var program = require('commander'); program .version('0.1.0') </pre> An Array is used for the value of a variadic argument. This applies to program.args as well as the argument passed to your action as demonstrated above.	<pre> .action(function (dir, otherDirs) { console.log('rmdir %s', dir); if (otherDirs) { otherDirs.forEach(function (odir) { console.log('rmdir %s', odir); }); } } }); program.parse(process.argv); </pre>	<pre> #!/usr/bin/env node /** * Module dependencies. */ var program = require('commander'); program .version('0.1.0') .command('rmdir <dir> [otherDirs...]',) </pre>	Variadic arguments <pre> console.log('size: %j', program.size); console.log('drink: %j', program.drink); program.drink(); </pre> The last argument of a command can be variadic, and only the last argument. To make an argument variadic you have to append ... to the argument name. Here is an example:
<pre> .arguments('<cmd> [env]') .action(function (cmd, env) { cmdValue = cmd; envValue = env; }); program.parse(process.argv); if (typeof cmdValue === 'undefined') { console.error('no command given!'); process.exit(1); } </pre>	<pre> console.log('command:', cmdValue); console.log('environment:', envValue "no environment given"); </pre> Angled brackets (e.g. <cmd>) indicate required input. Square brackets (e.g. [env]) indicate optional input. Git-style sub-commands <pre> // file: ./examples/pm var program = require('commander'); </pre>	<pre> program .version('0.1.0') .command('install [name]', 'install one or more packages') .command('search [query]', 'search with optional query') .command('list', 'list packages installed', {isDefault: true}) .parse(process.argv); </pre> When .command() is invoked with a description argument, no .action(callback) should be called to handle sub-commands,	otherwise there will be an error. This tells commander that you’re going to use separate executables for sub-commands, much like git(1) and other popular tools. The commander will try to search the executables in the directory of the entry script (like ./examples/pm) with the name program-command, like pm-install, pm-search. Options can be passed with the call to .command(). Specifying true for opts.noHelp will remove the subcommand from the
Custom help <pre> -b, --bbq Add bbq sauce -c, --cheese <type> Add the specified type of cheese [marble] -C, --no-cheese You do not want any cheese </pre> You can display arbitrary -h, --help information by listening for “-help”, Commander will automatically exit once you are done so that the remainder of your program does not execute causing undesired	<pre> \$./examples/pizza --help Usage: pizza [options] -h, --help output usage -V, --version output the version number -p, --peppers Add peppers -P, --pineapple Add pineapple </pre> An application for pizzas ordering	Automated –help The help information is auto-generated based on the information commander already knows about your program, so the following --help info is for free: <pre> some os version don't support this pattern. * Use the --harmony option when call the command, like node --harmony examples/pm publish. The --harmony option will be preserved when spawning sub-command process. </pre>	--harmony You can enable --harmony option in two ways: * Use #! /usr/bin/env node --harmony in the sub-commands scripts. Note generated help output. Specifying true for opts.isDefault will run the subcommand if no other subcommand is specified. If the program is designed to be installed globally, make sure the executables have proper modes, like 755.

behaviors, for example in the following executable "stuff" will not output when `--help` is used.

```
#!/usr/bin/env node
```

```
/**
 * Module dependencies.
 */
```

```
var program = require('commander');
```

program

33

```
.version('0.1.0')
.option('-f, --foo', 'enable some
    foo')
.option('-b, --bar', 'enable some
    bar')
.option('-B, --baz', 'enable some
    baz');
```

```
// must be before .parse() since
// node's emit() is immediate
```

```
program.on('--help', function(){
```

34

```
console.log('')
console.log('Examples:');
console.log(' $ custom-help --
  help');
console.log(' $ custom-help -h');
});
```

```
program.parse(process.argv);
```

```
console.log('stuff');
```

35

Yields the following help output when node script-name.js -h or node script-name.js --help are run:

Usage: custom-help [options]

Options:

```
-h, --help      output usage
information
-V, --version   output the version
number
-f, --foo       enable some foo
-b, --bar       enable some bar
```

36

·help(cb)

Output help information and exit immediately. Optional callback cb allows post-processing of help text before it is displayed.

Custom event listeners

You can execute custom actions by listening to command and option events.

```

    if (process.argv.slice(2).length) {
        program.outputHelp(make_red);
    }

    function make_red(txt) {
        return colors.red(txt); //display
        console
    }
}

```

```

        .version('0.1.0')
        .command('getstream [url]', 'get
        stream URL')
        .parse(process.argv);

    use something like:
    (e.g. if no command was provided), you can
    If you want to display help by default

```

```
-B, --enable-some-baz
    $ custom-help --help
    $ custom-help -h
```

```
program.on('option:verbose', function
  () {
    process.env.VERBOSE = this.verbose;
  });
```

```
// error on unknown commands
program.on('command:*', function () {
  console.error('Invalid command:
    %s\nSee --help for a list of
    available commands.',
    program.args.join(' '));
```

41

```
process.exit(1);
});
```

Examples

```
var program = require('commander');

program
  .version('0.1.0')
  .option('-C, --chdir <path>',
    'change the working
    directory')
```

```
.option('-c, --config <path>', 'set
    config path. defaults to ./
    deploy.conf')
.option('-T, --no-tests', 'ignore
    test hook');

program
    .command('setup [env]')
    .description('run setup commands for
        all envs')
    .option('-s, --setup_mode [mode]',
        "Which setup mode to use")
```

```
.action(function(env, options){
  var mode = options.setup_mode ||
    "normal";
  env = env || 'all';
  console.log('setup for %s env(s)
    with %s mode', env, mode);
});

program
  .command('exec <cmd>')
  .alias('ex')
```

44

MIT

License

More Demos can be found in the [examples](#) directory.

```
program.parse(process.argv);
```

```

console.log(' $ deploy exec
sequential');
console.log(' $ deploy exec
async');
}
};

program
.command('*')
.action(function(env){
  console.log('deploying "%s"',
    env);
});
};

```

```

.description('execute the given
remote cmd')
.option('-e', '--exec-mode<mode>',
    "which exec mode to use",
    {default: 'pwc'})
.option('--using %s',
    "log console messages using %s",
    {default: 'console.log',
    examples: ': , (, ), : , :'}
    )
}

```

48

IIW